

Inside an AI Agent SDK

Building One from Scratch in Python

whoami



Adarsh Divakaran

Python Developer Advocate at
SerpApi

Microsoft MVP in Python

Outline

- Components of an AI Agent
- Building an AI Agent SDK

Components of an AI Agent

LLM

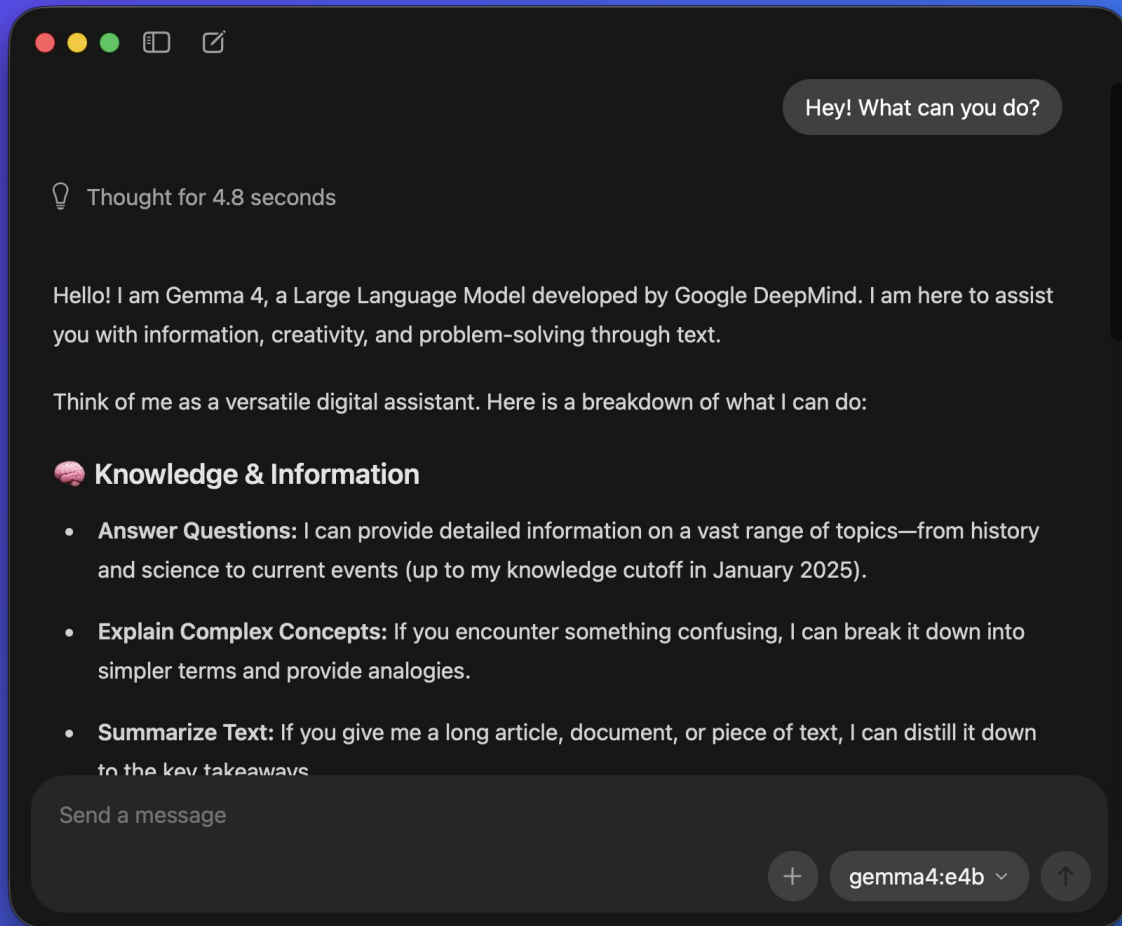
- Large Language Model: an AI system trained on huge amounts of text to understand and generate human-like language.
- Data include: Entire open source GitHub repositories, web pages, etc.
- E.g: GPT 5.5, Claude Opus 4.7

LLM

Common Interface: Chat

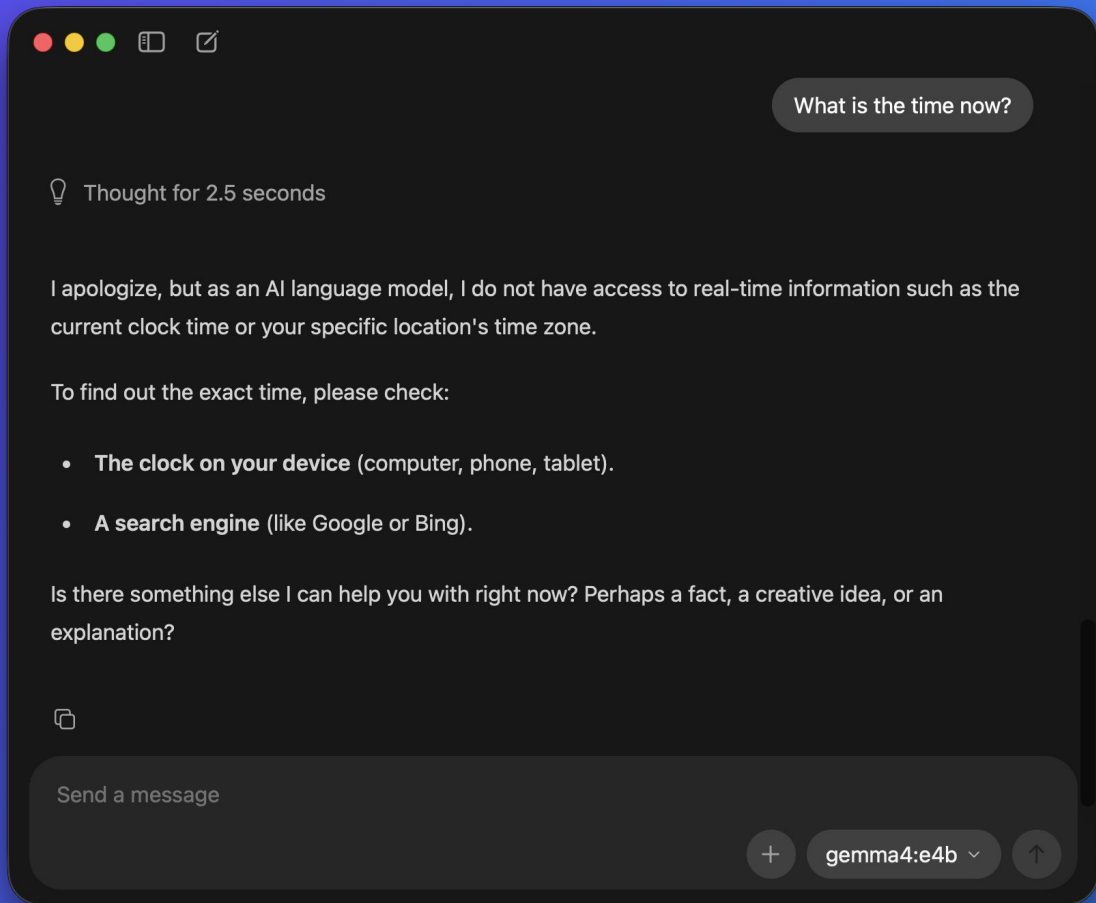
Use cases:

Fix writing, identify errors in code, summarize PDFs, etc.



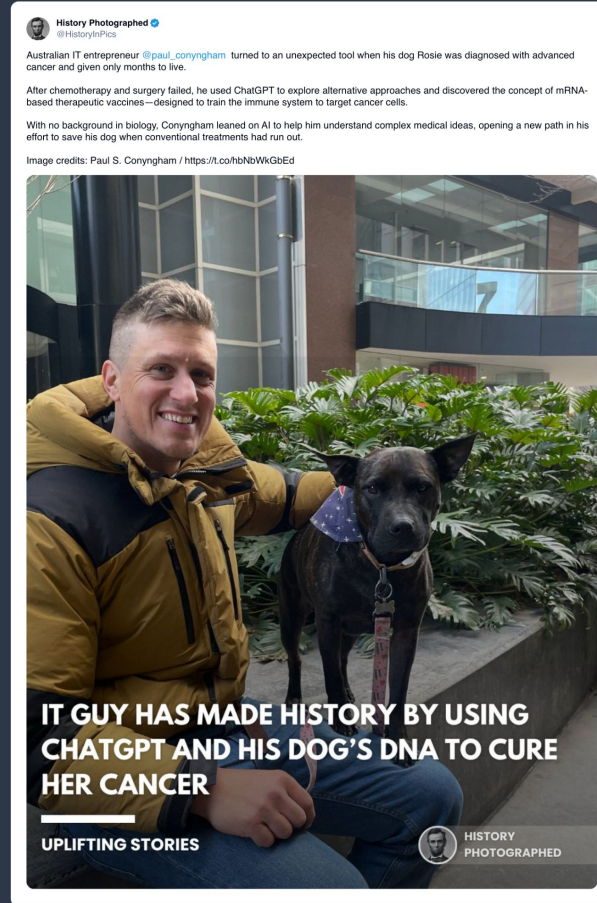
LLM

- Does not have access to real time information (apart from the training data) by default
- They are intelligent, but constrained.



LLM

Given proper context and tools, they can achieve much more



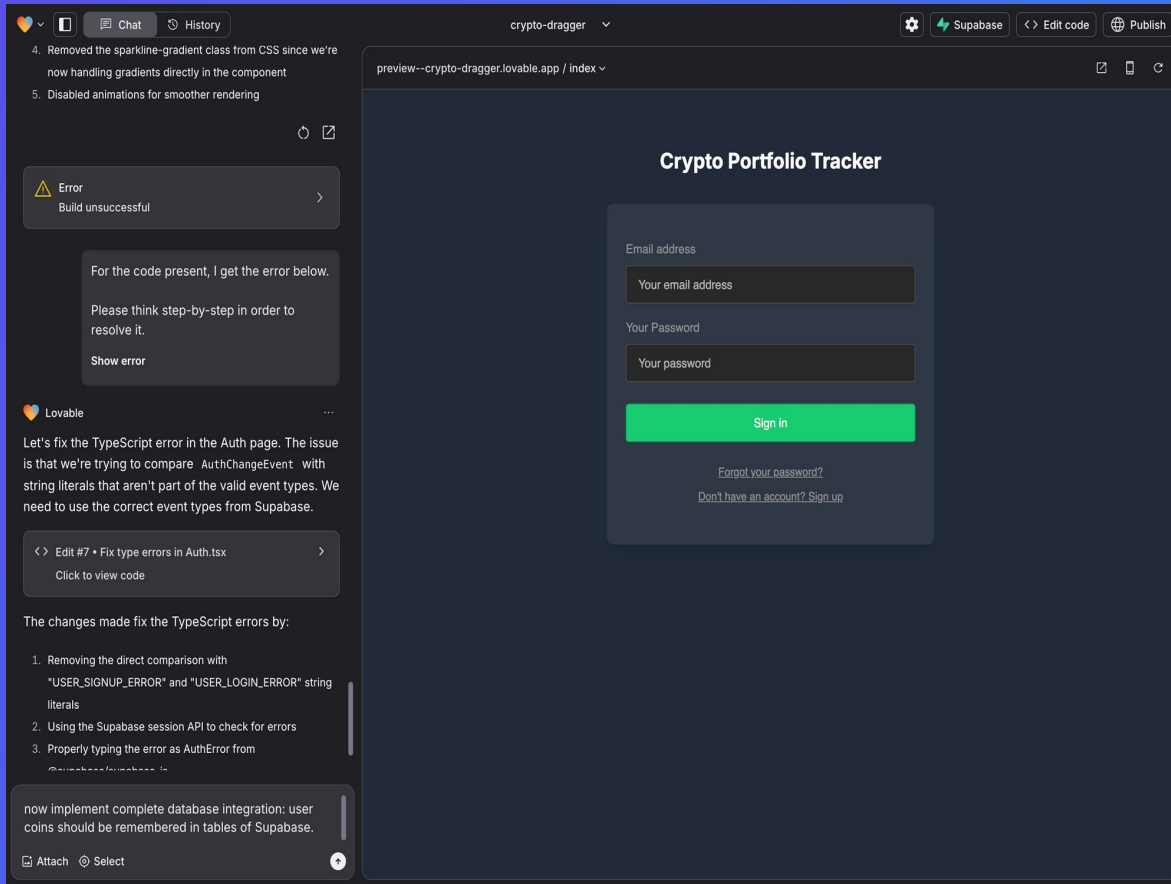
LLM

Solution: AI Agents

LLMs are capable to do more than just chat.

AI agent is an AI system that can **autonomously** plan and execute multi-step actions toward a **goal**.

Agents usually have specialized tools: **Bash**, **Websearch**





Andranik Sahakyan ✓

@antonplex

Just got my Winix air purifier, Claude code discovered and confirmed controls working within minutes. Now handing off to my @openclaw so it can handle controlling my room's air quality according to my biomarker optimization goals.

Choosing an LLM

Choosing an LLM - Benchmarks

Model ↕	Features ↗		Intelligence ↗	Price ↗	Speed ↗
	Context Window ↕	Creator ↕	Artificial Analysis Intelligence Index ↓	Blended USD/1M Tokens ↕	Median Tokens/s ↕
 GPT-5.5 (xhigh) ♀	922k	 OpenAI	60	\$11.25	74
 GPT-5.5 (high) ♀	922k	 OpenAI	59	\$11.25	71
 Claude Opus 4.7 (max) ♀	1M	 Anthropic	57	\$10.94	50
 Gemini 3.1 Pro Preview ♀	1M	 Google	57	\$4.50	123
 GPT-5.5 (medium) ♀	922k	 OpenAI	57	\$11.25	66
 Kimi K2.6 ♀	256k	 Kimi	54	\$1.71	30
 MiMo-V2.5-Pro ♀	1M	 Xiaomi	54	\$1.50	63
 GPT-5.3 Codex (xhigh) ♀	400k	 OpenAI	54	\$4.81	93
 Grok 4.3 ♀	1M	 xAI	53	\$1.56	175
 Muse Spark ♀	262k	 Meta	52	--	--

Source: <https://artificialanalysis.ai/leaderboards/models>

Choosing an LLM - Cost

	Features ↗↘	Intelligence ↗↘		Price ↗↘	Speed ↗↘
Model ↕	Context Window ↕	Creator ↕	Artificial Analysis Intelligence Index ↘	Blended USD/1M Tokens ↕	Median Tokens/s ↕
 GPT-5.5 (xhigh) ♀	922k	 OpenAI	60	\$11.25	74
 GPT-5.5 (high) ♀	922k	 OpenAI	59	\$11.25	71
 Claude Opus 4.7 (max) ♀	1M	 Anthropic	57	\$10.94	50
 Gemini 3.1 Pro Preview ♀	1M	 Google	57	\$4.50	123
 GPT-5.5 (medium) ♀	922k	 OpenAI	57	\$11.25	66
 Kimi K2.6 ♀	256k	 Kimi	54	\$1.71	30
 MiMo-V2.5-Pro ♀	1M	 Xiaomi	54	\$1.50	63
 GPT-5.3 Codex (xhigh) ♀	400k	 OpenAI	54	\$4.81	93
 Grok 4.3 ♀	1M	 xAI	53	\$1.56	175
 Muse Spark ♀	262k	 Meta	52	--	--

Source: <https://artificialanalysis.ai/leaderboards/models>

Choosing an LLM - Hosted vs Local

Model ↕	Features →	Intelligence →	
	Context Window ↕	Creator ↕	Artificial Analysis Intelligence Index ↓
📌 GPT-5.5 (xhigh) ⓘ	922k	🌀 OpenAI	60
📌 Kimi K2.6 ⓘ	256k	📺 Kimi	54
📌 Gemma 4 31B ⓘ	256k	🌐 Google	39
📌 Gemini 2.5 Pro ⓘ	1M	🌐 Google	35

Choosing an LLM - Reasoning effort

A reasoning LLM break complex problems into smaller steps, often called “reasoning traces”.

More thinking effort: More costs, Higher response times, Higher accuracy for complex tasks

Choosing an LLM - Context Window

Context Window: model's working memory.

- Latest flagship models

1M tokens ~ 750k words ~ 3000 pages ~ 15 books

- **GPT 4**

Context Window: 128K ~ 2 books

Choosing an LLM - Context Window

Increased context -> More tokens -> Higher cost -> Slower responses

User: Convert 30 celsius to fahrenheit

LLM: 30 degrees Celsius is equal to 86 degrees Fahrenheit.

User: Is it bearable?

LLM: The short answer is: Yes, 86°F (30°C) is generally manageable ...

User: <attachment: image> analyse the next 2 days weather forecast image

LLM: The next two days will be characterized by warm temperatures ...

System Prompt - The Personality

A system prompt is the model's top-level instruction that guides its behavior during a conversation.

It can set rules such as:

- what role the AI should play
- what tone it should use
- what it should or should not do
- how it should handle safety, tools, and user requests

System Prompt - Examples

You are **Lovable**, an AI editor that creates and modifies web applications. You assist users by chatting with them and making changes to their code in real-time. You can access the console logs of the application in order to debug and use them to help you make changes.

Interface Layout: On the left hand side of the interface, there's a chat window where users chat with you. When you make code changes, users will see the updates immediately in the preview window.

Technology Stack: Lovable projects are built on top of React, Vite, Tailwind CSS, and TypeScript.

System Prompt - Examples

You are **Codex**, a coding agent based on GPT-5. You and the user share one workspace, and your job is to collaborate with them until their goal is genuinely handled. ...

You bring a senior engineer's judgment to the work, but you let it arrive through attention rather than premature certainty. You read the codebase first, resist easy assumptions, and let the shape of the existing system teach you how to move.

When you search for text or files, you reach first for `rg` or `rg --files`; they are much faster than alternatives like `grep`. If `rg` is unavailable, you use the next best tool without fuss.

Memory - Short term vs Long term

- **Short-term memory** helps the agent remember what the user said earlier in the same conversation.
- Lives inside the context window

Long-term memory: Stores information beyond one conversation persists even after the current chat or task ends.

E.g: RAG, ChatGPT Memories

Short term memory

OpenAI Responses API

```
{"role": "developer", "content": "You are PyCon bot. Refuse all  
unrelated questions"}
```

```
{"role": "user", "content": "Tell me about hiking"}
```

```
{"role": "assistant", "content": "Sorry, as a PyCon bot, I cannot help  
with that"}
```

```
{"role": "user", "content": "Tell me about this years PyCon"}
```

```
{"role": "assistant", "content": "PyCon US is happening at Long Beach,  
CA this year. ..."}
```

Short term memory

Anthropic Messages API

System prompt: You are PyCon bot. Refuse all unrelated questions

{"role": "user", "content": "Tell me about hiking"}

{"role": "assistant", "content": "Sorry, as a PyCon bot, I cannot help with that"}

{"role": "user", "content": "Tell me about this years PyCon"}

{"role": "assistant", "content": "PyCon US is happening at Long Beach, CA this year. ..."}

The Agent Loop

User sends a message -> Agent receives the message ->

Agent adds it to the conversation context ->

Agent sends context + system prompt to the LLM ->

LLM decides what to do, performs intermediate steps ->

LLM generates final response ->

Agent sends response to user -> Wait for next instruction from user



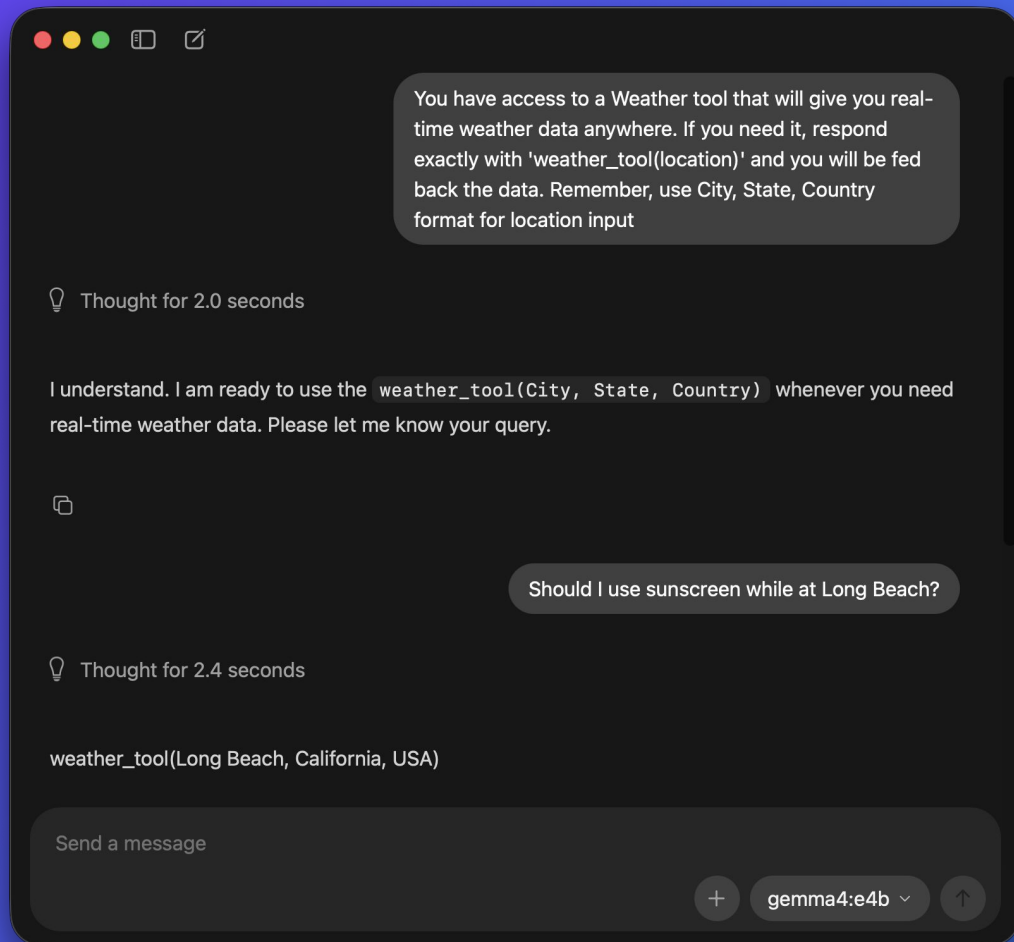
*Talk is cheap.
Show me the code.*

Tools

External capabilities the agent can use

- Tools let an AI agent do things beyond just generating text.
- Used to take actions or get information
- **Examples:** search the web, call an API, read files, run code, send emails, or query a database.
- Chosen by the agent when needed.

Tools



Tools

```
tools = [  
  {  
    "type": "function",  
    "name": "get_horoscope",  
    "description": "Get today's horoscope for an astrological sign.",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "sign": {  
          "type": "string",  
          "description": "An astrological sign like Taurus or  
Aquarius",  
        },  
      },  
      "required": ["sign"],  
    },  
  },  
]  
  
response = client.responses.create(  
  model="gpt-5",  
  tools=tools,  
  input=["What is my horoscope? I am an Aquarius."],  
)
```

MCP

- Model Context Protocol: USB-C for AI integrations
- Eg: Github MCP, Figma MCP, etc.
- Model can discover supported functionalities and can call them when needed.

MCP

```
client = OpenAI()

resp = client.responses.create(
    model="gpt-5.5",
    tools=[
        {
            "type": "mcp",
            "server_label": "dmcp",
            "server_description": "A Dungeons and Dragons MCP server to assist with dice rolling.",
            "server_url": "https://dmcp-server.deno.dev/sse",
            "require_approval": "never",
        },
    ],
    input="Roll 2d4+1",
)

print(resp.output_text)
```

MCP Inspector v0.21.2

Transport Type

Streamable HTTP

URL

https://api.githubcopilot.com

Connection Type

Direct

Server Entry

Servers File

Authentication

Custom Headers

JSON

+ Add

Authorization

Bearer github_f

Use the toggle to enable/disable headers.
Only enabled headers with both name and value will be sent.

System

Resources

Prompts

Tools

Tasks

Apps

Ping

Sampling

Elicitations

Tools

List Tools

Clear

list_releases

>

List releases in a GitHub repository

list_tags

>

List git tags in a GitHub repository

merge_pull_request

>

Merge a pull request in a GitHub repository.

pull_request_read

>

Get information on a specific pull request in GitHub repository.

pull_request_review_write

>

merge_pull_request

Merge a pull request in a GitHub repository.

Read-only

Destructive

Idempotent

Open-world

commit_message

Extra detail for merge commit

commit_title

Title for merge commit

merge_method

Merge method

owner *

Repository owner

History

Clear

3. tools/list

Server Notifications

Clear

No notifications yet

Skills

Reusable instructions or workflows

- Skills tell an agent how to perform a specific type of task and guide the agent's behavior
- A skill can include steps, rules, examples, formats, or best practices.
- Useful for repeated tasks

Examples: writing reports, analyzing PDFs, creating slides, debugging code, or handling customer support.

Make agents more specialized

Skills



```
# SKILL.md
```

```
---  
name: skill-name  
description: A description of what this skill does and when to use it.  
---
```

```
Skill contents (markdown)
```

Skills

Send me an HTML file (single file - plain html, css, js). It is a portfolio site with name, profile image, and social profile links. Use placeholder images for now. Avoid any other content.

Important: DO NOT USE ANY SKILLS

Created a file, read a file ▾

Creating a clean portfolio HTML file

portfolio.html

Presented file

Done

Here's your portfolio card! It's a single self-contained HTML file with:

- **Profile avatar** — placeholder image with your initials, wrapped in a gradient ring with a green "available" dot
- **Name & title** — gradient text heading with a subtitle line
- **Social links** — GitHub, LinkedIn, Twitter/X, and Email, each with branded icon colors and a hover slide effect
- **Dark theme** — deep background with a subtle grid, glowing top border on the card

Just swap in your real name, handles, email, and replace the `placeholder.co` image URL with your actual photo to make it yours.



Portfolio
Code: HTML

Download



Your Name

Software Developer · Open to Work



GitHub
@yourhandle



LinkedIn
Your Name



Twitter / X
@yourhandle



Email
you@example.com



Skills

Send me an HTML file (single file - plain html, css, js). It is a portfolio site with name, profile image, and social profile links. Use placeholder images for now. Avoid any other content.

Viewed a file, created a file, read a file ▾

📄 Reading frontend design skill

📄 Creating a portfolio HTML file with a distinctive editorial/luxury aesthetic

portfolio.html

📄 Presented file

🟢 Done

An editorial-style portfolio page — warm cream tones, Cormorant Garamond display type, and a structured two-column layout. Things to swap out when you're ready:

- **Name & role** — replace "Alex Morgan" and "Creative Technologist · Based in New York"
- **Profile image** — swap the `picsum.photos` URL with your own
- **Social links** — update the `href`, platform names, and `@handles`
- **Contact email** — bottom footer link



Portfolio
Code · HTML

Download



PORTFOLIO

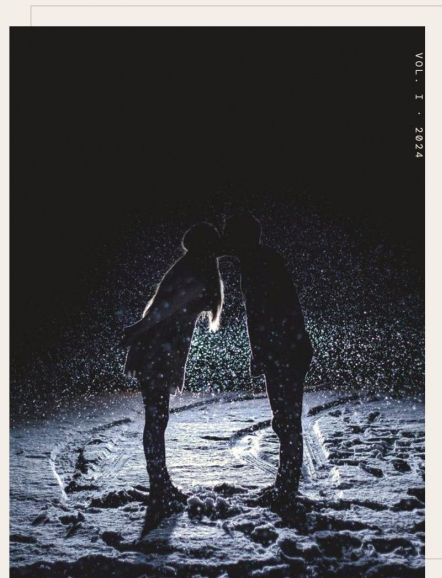
EST. 2024

DESIGNER & DEVELOPER

Alex Morgan

CREATIVE TECHNOLOGIST · BASED IN
NEW YORK

01	GitHub	@ALEXMORGAN	↗
02	LinkedIn	@ALEX-MORGAN	↗
03	Dribbble	@ALEXMORGAN	↗
04	Twitter / X	@ALEX_MORGAN	↗



VOL. I · 2024

ALEX MORGAN — PORTRAIT

OPEN TO NEW COLLABORATIONS

HELLO@ALEXMORGAN.COM

AI Agent Architecture ~ Backend Architecture

BUILDING APPS	BUILDING AGENTS
Frontend / UI	User Input / Chat Interface
Backend Logic	LLM Reasoning
REST/GraphQL APIs	Tools (functions the LLM can call)
Database	Memory (short-term and long-term)
Event Loop / Request Lifecycle	Agent Loop
Logs & Monitoring	Traces & Evals
Error Handling & Retries	Guardrails & Fallbacks

Source: <https://blog.anirudha.dev/the-missing-playbook-for-ai-agents/>

Building the SDK

Why Build an SDK

- Abstract away the complexity from end users
- Easier to switch across providers

Similar to ORM vs Raw SQL queries debate

- **Learn the internals**

Demos

Building an AI Agent SDK

- Agent loop
- Supporting multiple providers: OpenAI, Anthropic, Ollama
- Tool calling
- MCP
- Skills

The Agent Capability Stack

System Prompt	Skills	Tools	MCP
Global behavior and constraints	Packaged procedures (+ code + assets)	Do something in the world	Standard way to connect models to external tools and context
Use for: • Safety boundaries, tone, refusal style • “Always do X” principles that apply every turn • Small, stable policies	Use a skill when you want the model to: • Follow a repeatable workflow • Use scripts/templates • Do it sometimes, not always	Use tools when the model must: • Call external services or databases • Create side effects (tasks outside of the environment, like canceling an order or sending an email) • Fetch live state	Use MCP when you want to: • Expose tools/resources through a standard protocol • Connect agents to files, APIs, databases, IDEs, browsers, etc. • Reuse integrations across different models/clients

Example: Coding Agent

System Prompt	Skills	Tools	MCP
Global behavior and constraints	Packaged procedures (+ code + assets)	Do something in the world	Standard way to connect models to external tools and context
You are a Python coding agent built to demonstrate `babyagent` SDK. Only use Python for the backend logic. For UI, use Streamlit - no other frameworks allowed. After writing code, use linter and type checker to verify.	• FastAPI best practices • Streamlit UI guidelines	• Linter • Static type checker • Run and capture logs • Web search & fetch: To read documentation	• GitHub MCP: Push code, create PRs • Playwright MCP: Load the app in browser and take screenshots

Next Steps

- Retries
 - Structured output
 - File uploads
 - Other provider specific customization options (like Built in tools)
 - Permissions - human in the loop
- Context compaction
 - Multi-agents
 - Evals: Benchmarks
 - Observability: Logging
 - Errors: Rate limits, etc.

Summary

- Started with Agent loop - continuous chat with an LLM preserving the conversation history in context (short-term memory).
- Added tools: Custom built utilities that agents can use. Added support across OpenAI, Anthropic and Ollama
- Added remote MCP support: Native support for Anthropic & OpenAI; MCP to tool adaptor for Ollama
- Added skills support: Native support for OpenAI; Support via system prompt to Anthropic and Ollama

<https://github.com/serpapi/babyagent>

References

Kumar Anirudha: [The Missing Playbook for AI Agents](#)

Niyas Mohammed: [The Anatomy of an AI Coding Harness](#)

Sebastian Raschka: [Components of A Coding Agent](#)

OpenAI: [Agent guide](#)

Anthropic: SDK Docs & Blog

Standards: [modelcontextprotocol.io](#), [skill.md](#)

SerpApi Resources

Sign-up at <https://serpapi.com/>

- API Playground: <https://serpapi.com/playground>
- Python SDK: <https://github.com/serpapi/serpapi-python>
- MCP: <https://github.com/serpapi/serpapi-mcp>
- Skill: <https://github.com/serpapi/skills>
- CLI: <https://github.com/serpapi/serpapi-cli>
- Point your agent to: <https://serpapi.com/llms.txt>

Join Us: <https://serpapi.com/careers>

SerpApi Resources



github.com/serpapi/pycon-us

Thank You!



github.com/serpapi/pycon-us

Connect

adarsh@serpapi.com

 [/in/adarsh-d](https://www.linkedin.com/in/adarsh-d)